

Architecture of Database Management System

Early Two Level Architecture

- Early proposal for a standard terminology and general architecture for database systems was produced in 1971 by the DBTG (Data Base Task Group)
- Appointed by the Conference on Data Systems and Languages (CODASYL, 1971).
- Recommend two level approach.

1. Schema
2. Subschema

Schema

4

- A map of the overall logical structure or description of the database .
- It gives the names of the entities, attributes and specifies the relationships among the entities.
- A schema is defined as an outline or a plan that describes the records and relationships existing at the particular level.
- The schema is sometimes called the **intension of the database**, while an instance is called an **extension (or state)** of the database.

Subschema

5

- A map of application programmer's view of the data he/she uses.
- It is a subset of the schema having the same properties that a schema has. It allows the user to view only that part of the database that is of interest to him. They are values of a table at a particular time.

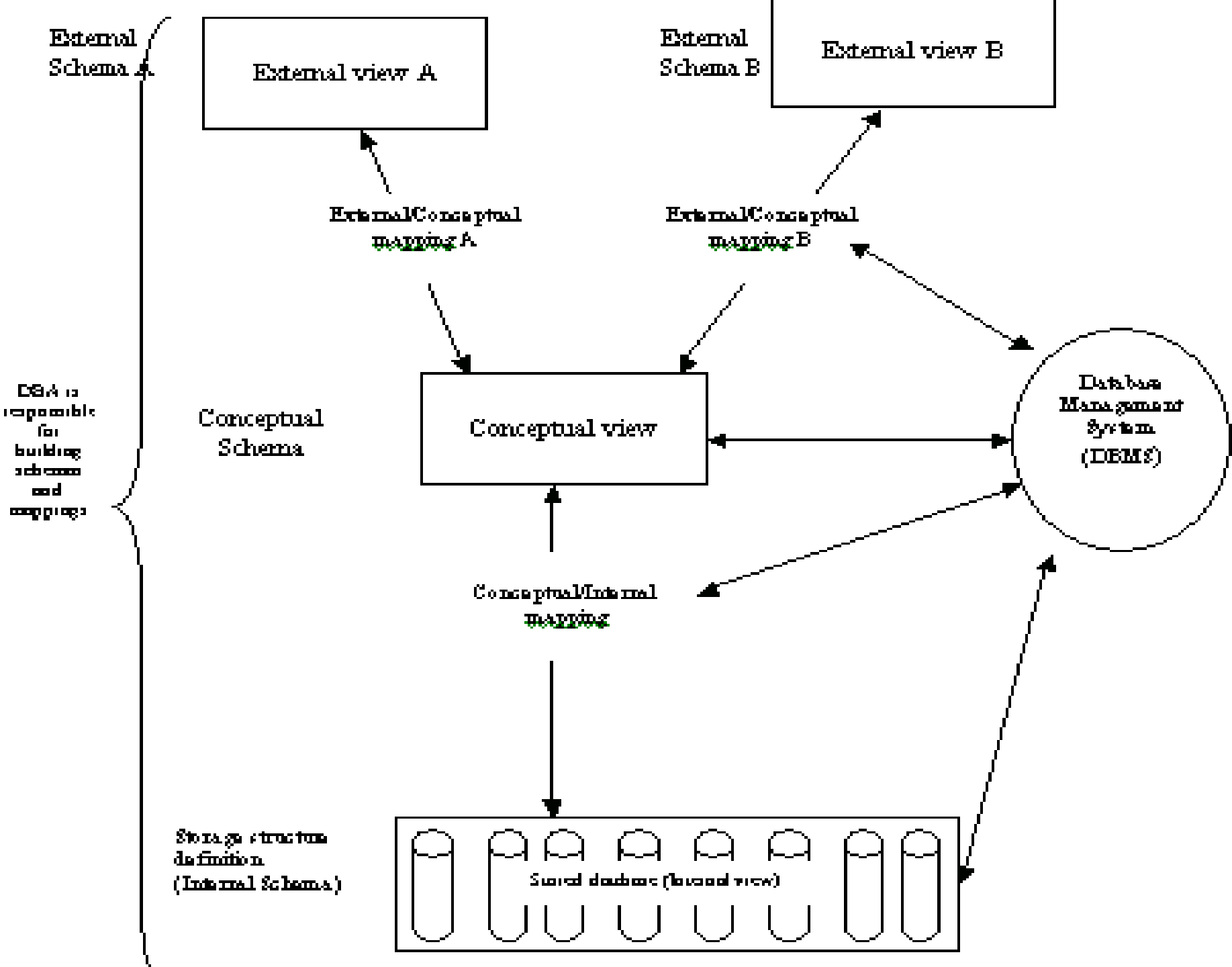
Three Level Architecture

There are following three levels or layers of DBMS architecture:

- **External Level**

- **Conceptual Level**

- **Internal Level**



External Level or View level

- It is the users' view of the database. This level describes that part of the database that is relevant to each user.
- For example, one user may view dates in the form (day, month, year), while another may view dates as (year, month, day).

Conceptual Level or Logical level

- It is the community view of the database.
- This level describes **what data is stored in the database** and the relationships among the data.
- It represents:
 - ☐ All entities, their attributes, and the relationships among them;
 - ☐ The constraints on the data;
 - ☐ Security and integrity information.

Internal Level or Storage level or Physical Level

- It is the **physical representation** of the database on the computer.
- This level describes **how the data is stored** in the database.
 - The internal level is concerned with such things as:
 - ☐ Storage space allocation for data and indexes;
 - ☐ Record descriptions for storage (with stored sizes for data items);
 - ☐ Record placement;
 - ☐ Data compression and data encryption techniques.

¹
Roll Number
Name
Address
Book Number
Date of Issue
Date of Return

¹
Roll Number
Name
Fee

Conceptual Level

Field Name	Data Type
Roll Number	Number(6)
Name	Character(20)
Address	Character(40)
Book Number	Number(6)
Date of Issue	Date
Date of Return	Date
Fee	Number(5+2)

Internal Level

Stored Item	Length
Roll Number	Type=Byte(6), Offset=0
Name	Type=Byte(20), Offset=6
Address	Type=Byte(40), Offset=26
Book Number	Type=Byte(6), Offset=66
Date of Issue	Type=Byte(8), Offset=72
Date of Return	Type=Byte(8), Offset=80
Fee	Type=Byte(8), Offset=88

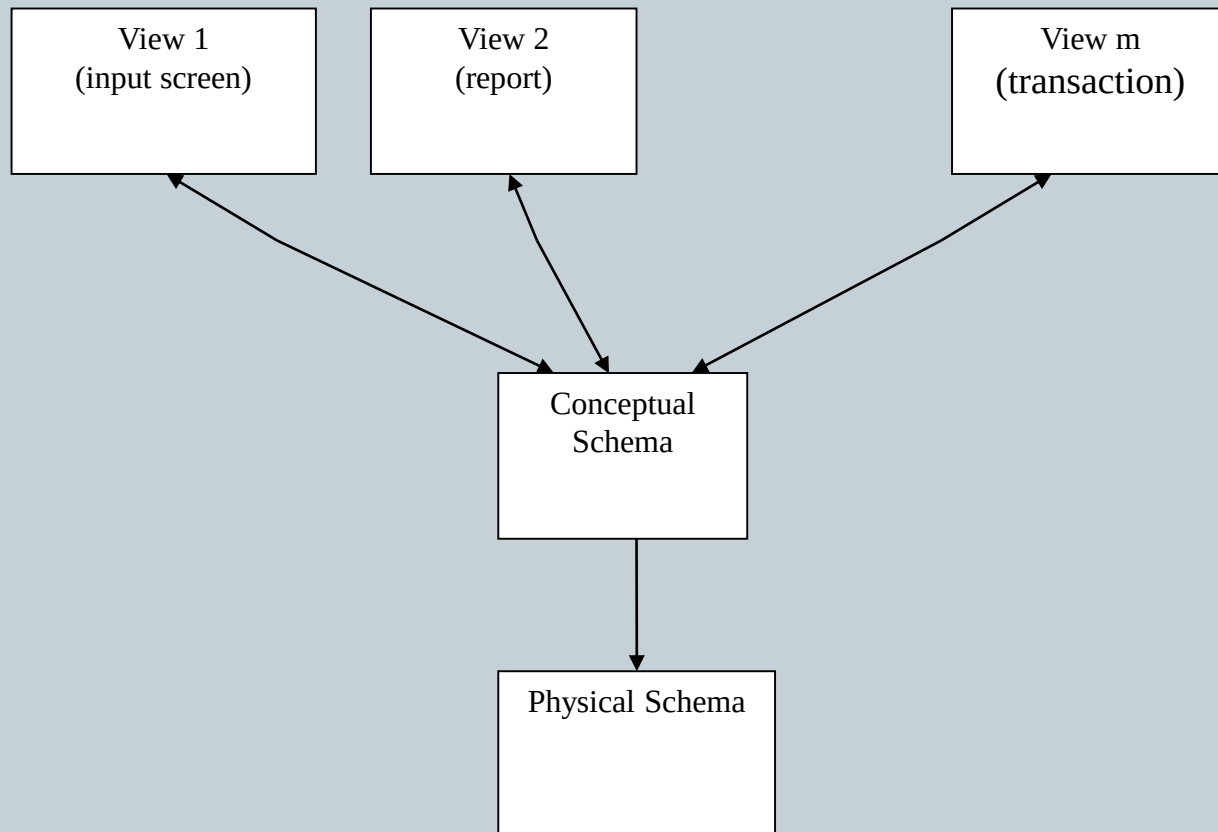
Mapping between Views

External/Conceptual Mapping:

A mapping between the external and conceptual views gives the correspondence among the records and the relationships of the external and conceptual views.

Conceptual/Internal Mapping:

Conceptual schema is related to the internal schema by the conceptual/internal mapping. This enables the DBMS to find the actual record or combination of records in physical storage that constitute a logical record in conceptual schema.



Data Independence-Achievement of Layered Architecture of DBMS

There are two kinds of data independence:

- Logical data independence
- Physical data independence

Logical data independence

- Logical data independence indicates that the conceptual schema can be changed without affecting the existing external schemas. Ability to modify the conceptual schema without causing application programs to be re-written.
- Modifications at conceptual level are necessary whenever the logical structure of the database is altered.

Physical data independence

- Physical data independence indicates that the physical storage structures or devices could be changed without affecting conceptual schema. Ability to modify the physical schema without causing application programs to be re-written.
- Modifications at physical level are occasionally necessary in order to improve performance.
- **Note:** logical data independence is more difficult to achieve than physical data independence, because application programs are heavily dependent on the logical structure of the data, they access.

Ex: Thapar Course Registration Database

- **Conceptual schema:**

student (st_id: string, name: string, birthdate: date, gender: character,
pa_l1: character, pa_l2: character, pa_l3: character,
la_l1: character, la_l2: character, la_l3: character,
cugp: integer, cup: integer, cgpa: real)

course (courseno: string, cname: string, credits: integer,
lect_h: integer, tut_h: integer, pract_h: integer)

instructor (instructor_id: string, name: string, design: string, group:
string,
specialization: string, chamber_no: string, tel_no: integer)

course_info (course_no: string, enrollment: integer, sections: integer)

st_course_reg (stid: string, course_no: string, section: integer,
sem: integer, year: string, grade: string)

inst_course_reg (instructor_id: string, courseno: string, section: integer,
sem: integer, year: string)

Ex: Thapar Course Registration Database

• External Schema (View):

course-allotment (instructor_id: string, group: string,
specialization: string,
course_no: string, section: integer)

course-reg (st_id: string, course_no: string,
section: integer)

grades (st_id: string, course_no: string,
grade: string)

Ex: Thapar Course Registration Database

19

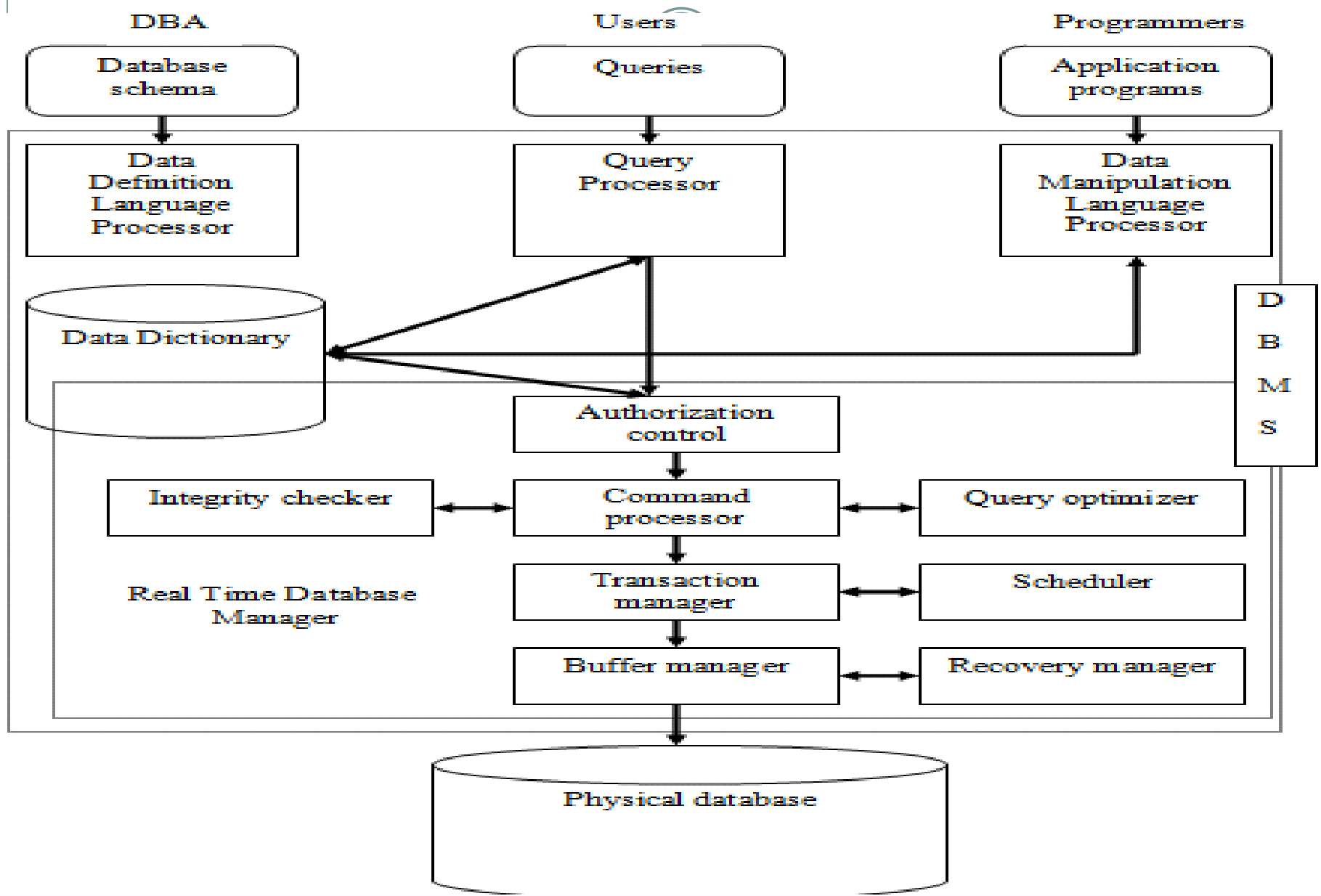
- **Physical schema:**
 - Relations stored as unordered files.
 - Indexes on column(s) of relations (tables).

Metadata or Data Dictionary

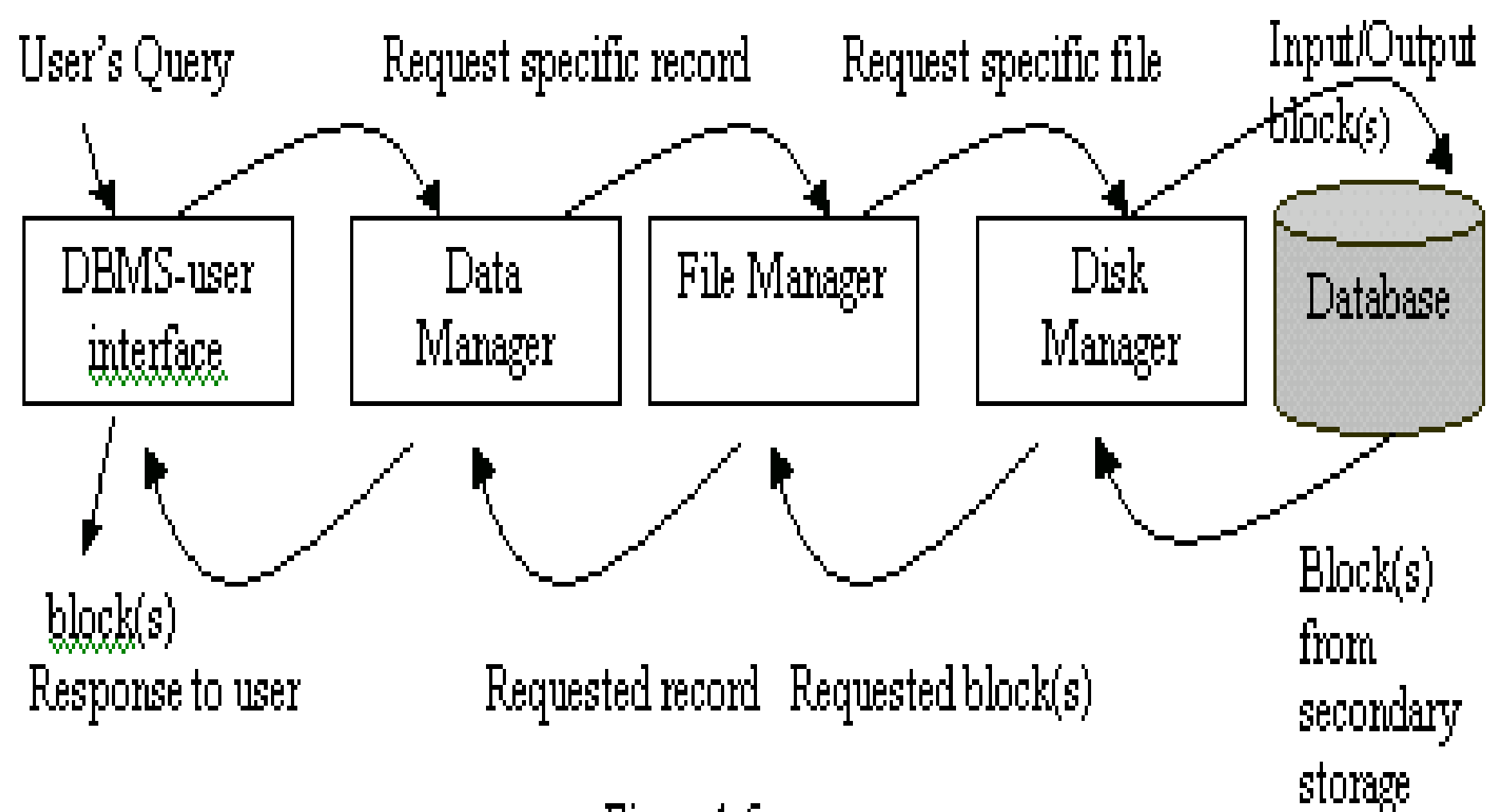
20

- A metadata (also called the data dictionary) is the data about the data.
- Data dictionary is a file that contains meta data. i.e., data about data.
- Data dictionary file is consulted before actual data is read or modified in the database.

Components of a DBMS



The Procedure for Database Access



END